

An Analysis and Simulation of Defending Tracking Control Neural Network for Known Affine Discrete-Time Nonlinear Systems Against Adversarial Attacks

Mentee: Ablasse Kingcaid-Ouedraogo,

Mentor: Dr. Pavlo Tymoshchuk

the date of receipt and acceptance should be inserted later

Abstract A neural network (NN) tracking control for discrete-time nonlinear systems with known internal dynamics is designed. The adversarial attack on the tracking control NN is described by Gaussian distributed additive stochastic noise. The adversarially perturbed system and the corresponding closed-loop tracking dynamics are derived. A functional block-diagram of the controlled nonlinear system is presented. Computer simulations of the controller operation under nominal and adversarial conditions are provided. A C++ implementation of the simulation is compared against a MATLAB reference to confirm numerical consistency.

Keywords control systems, neural networks, adversarial attacks

1 Introduction

The tracking control problem is one of the most fundamental practical problems in control theory, where the goal is to design a controller that allows a system to follow a desired trajectory or reference signal over time. In recent years, neural networks have been increasingly used in control systems due to their ability to approximate complex nonlinear functions and adapt to changing conditions. However, the use of neural networks in control systems also introduces new vulnerabilities, particularly to adversarial attacks. Small, practically undetectable perturbations in the input of a neural network can lead to outsized adverse effects on the output, which can be particularly problematic in control systems where safety and reliability are paramount.

A tracking control problem can be formulated as designing such a system whose output approximately repeats a given time-varying reference trajectory [1]. The defense of a tracking control neural network against adversarial perturbations requires that, under the influence of such perturbations, tracking performance remains acceptable within certain conditions. These conditions vary and are dependent upon the designed control law's inherent robustness, the magnitude and nature of the perturbation and the performance metrics required of the system for safe operation. The channel of attack chosen for this analysis is the error dynamics of the tracking control system described in Equation 3 [1].

A continuous stirred tank reactor (CSTR) system is chosen as a common close analog to compare and confirm the results of the analysis. The CSTR is a widely studied system in control theory, making it suitable for comparison with competitive analogs.

System Model

We consider a known affine discrete-time nonlinear system described by the following difference equation [1]:

$$x(k+1) = x(k) + \tau[f(x(k)) + g(x(k))u(k)], \quad x(1) = x_1 \quad (1)$$

where $x(k) \in \mathbb{R}^n$ is the discrete-time state variable, $f(x(k)) \in \mathbb{R}^n$ is the nonlinear function of the system's internal state dynamics, $g(x(k)) \in \mathbb{R}^{n \times m}$ is the nonlinear function that defines the effect of the control input on the state dynamics, $u(k) \in \mathbb{R}^m$ is the control input applied to the system, and $\tau > 0$ is the discrete-time step.

Control Function

The control $u(k)$ can be determined from Equation 1 considering the tracking error by the following control equation [1]:

$$u(k) = \begin{cases} g(x(k))^{-1}(-f(x(k)) + \Delta r(k) - \beta \text{sgn}(x(k) - r(k))) & \text{if } g(x(k)) \neq 0 \\ 0 & \text{if } g(x(k)) = 0 \end{cases} \quad (2)$$

such that, given a reference trajectory $r(k)$, the system Equation 1 seeks to minimize the closed-loop tracking error. The following associated difference equation with variable structure describes the closed-loop tracking error dynamics [1]:

$$e(k+1) = e(k) - \beta \text{sgn}(e(k))$$

where:

$$\text{sgn}(e(k)) = \begin{cases} 1 & \text{if } e(k) > 0 \\ 0 & \text{if } e(k) = 0 \\ -1 & \text{if } e(k) < 0 \end{cases} \quad (3)$$

and

$$e(k) = x(k) - r(k)$$

Closed-Loop Tracking Control

The difference equation Equation 1 and control equation Equation 2 describe the full closed-loop dynamics. Substituting $u(k)$ from Equation 2 into Equation 1 gives the closed-loop state equation [1]:

$$x(k+1) = x(k) - \beta \text{sgn}(x(k) - r(k)) + \Delta r(k)$$

where:

$$\Delta r(k) = r(k+1) - r(k) \quad (4)$$

Functional Block-Diagram of the Nominal Controlled Nonlinear System

A functional block-diagram of the controlled nonlinear system described by Equation 1 and Equation 2 is presented in Figure 1. The diagram includes the blocks of discrete-time summation $+$, sign activation function S , amplification β , nonlinear functions $f(x)$ and $g(x)$, NN solver of system of linear algebraic equations LAESS, reference r , state variable $x(k)$ with initial value $x(1)$, and control input $u(k)$. As one can see in Figure 1, the NN consists of $3n$ adders, n controlled switches, n amplifiers, n digital differentiators, n blocks of nonlinear functions $f(x)$, $n \times m$ blocks of nonlinear functions $g(x)$, and n NN solvers of a systems of linear algebraic equations [1].

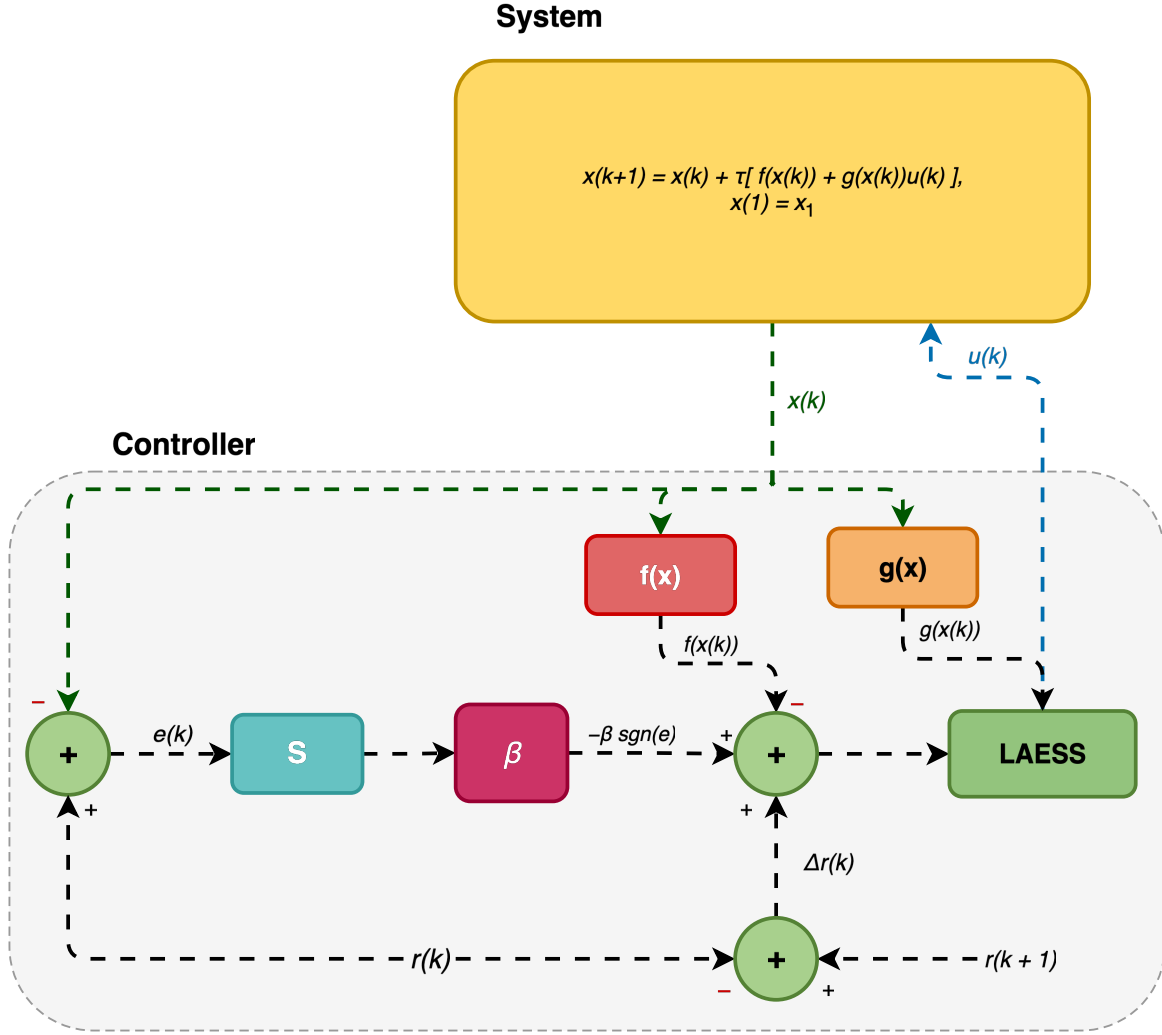


Fig. 1: Functional block-diagram of the controlled nonlinear system.

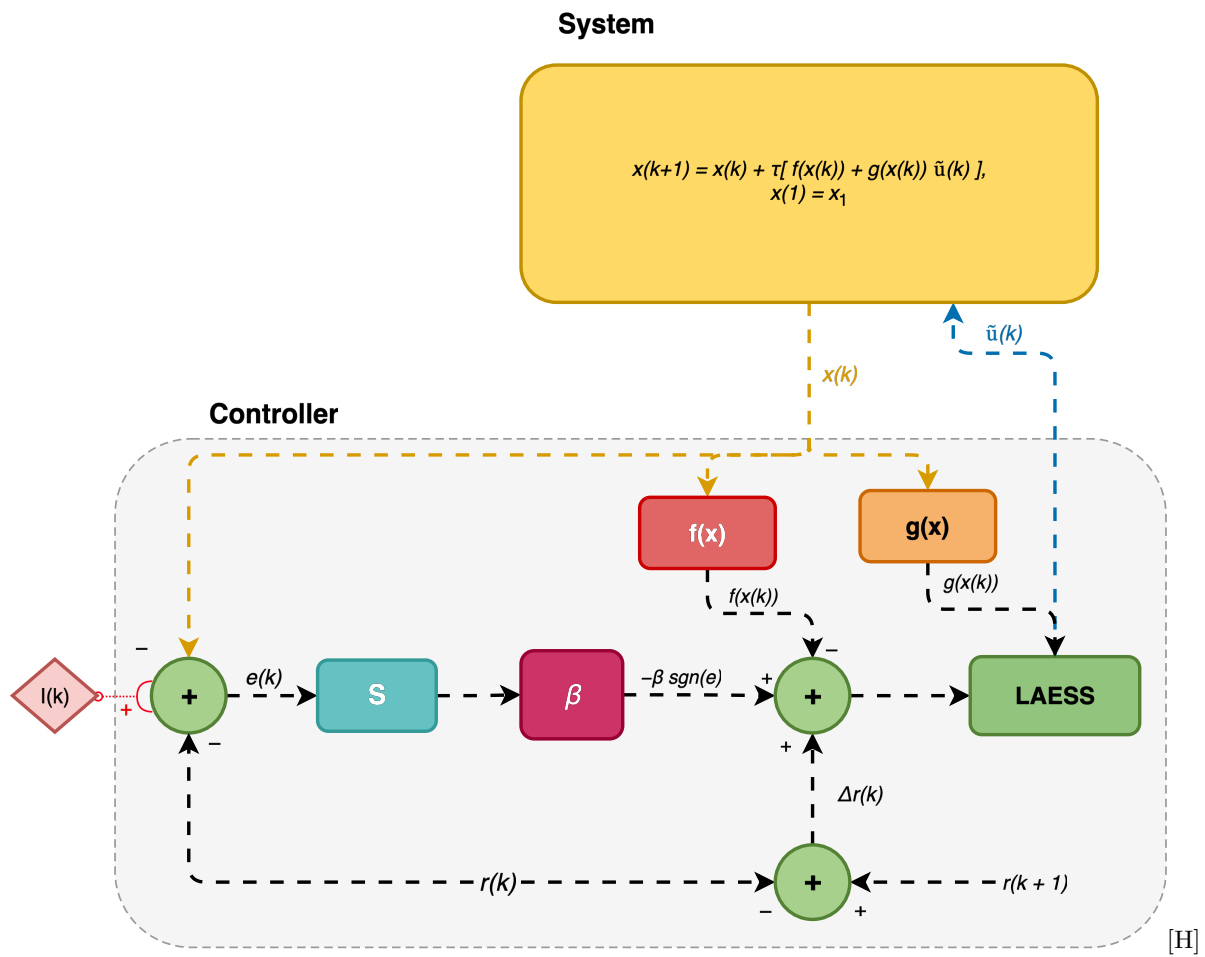
2 Adversarial Attacks

The current state-of-the-art method for adversarial attacks against neural networks is the projected gradient descent (PGD) attack [2], an iterative method that seeks to find the optimal perturbation maximizing the loss function of the neural network. The simulation and defense of a neural network using the PGD method will be left for future analysis. In this paper, we examine an alternative method of attack simulation using additive Gaussian noise perturbation [3] on the error channel, $e(k)$. The mathematical representation of the adversarially perturbed system, considering Equation 1, can be given by:

$$\begin{aligned}
 x(k+1) &= x(k) + \tau[f(x(k)) + g(x(k))\tilde{u}(k)] \\
 \text{where} \\
 \tilde{u}(k) &= (g(x(k)))^{-1}(-f(x(k)) + \Delta r(k) - \beta \text{sgn}(\tilde{e}(k))) \\
 \text{and} \\
 \tilde{e}(k) &= (r(k) - x(k)) + \epsilon(k) \\
 \text{and} \\
 \epsilon(k) &\sim \mathcal{N}(0, \mathcal{I}d_n) \text{ is the standard normal.}
 \end{aligned} \tag{5}$$

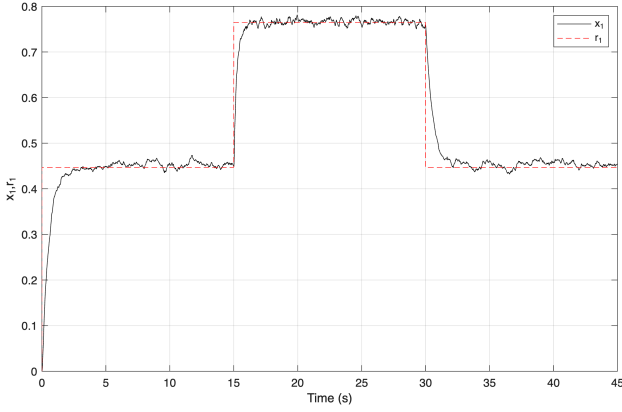
Substituting the control function from Equation 2 into the perturbed system model gives the adversarially perturbed closed-loop state equation:

$$\begin{aligned}
 x(k+1) &= x(k) - \beta \text{sgn}((x(k) - r(k)) + \epsilon(k)) + \Delta r(k) \\
 \text{where} \\
 \epsilon(k) &\sim \mathcal{N}(0, \mathcal{I}d_n)
 \end{aligned} \tag{6}$$

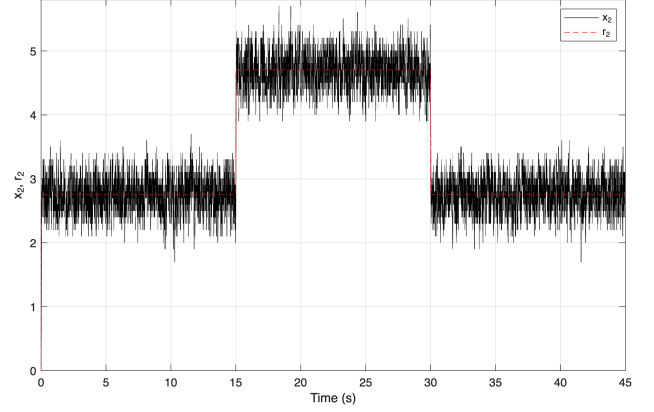


2.1 Simulation of Attacked CSTR System

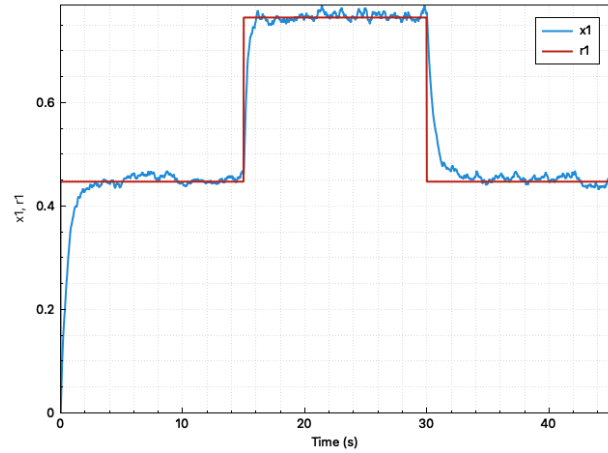
The trajectories of state variables $x_1(k)$ and $x_2(k)$ under adversarial perturbation are presented in Figure 2. Figure 2a and Figure 2b show the MATLAB reference implementation results for conversion x_1 and temperature x_2 , respectively. Figure 2c and Figure 2d show the corresponding C++ port results. Both implementations exhibit degraded tracking performance once the stochastic perturbation ($\sigma = 2$) is introduced, confirming that the adversarial attack model produces consistent effects across platforms.



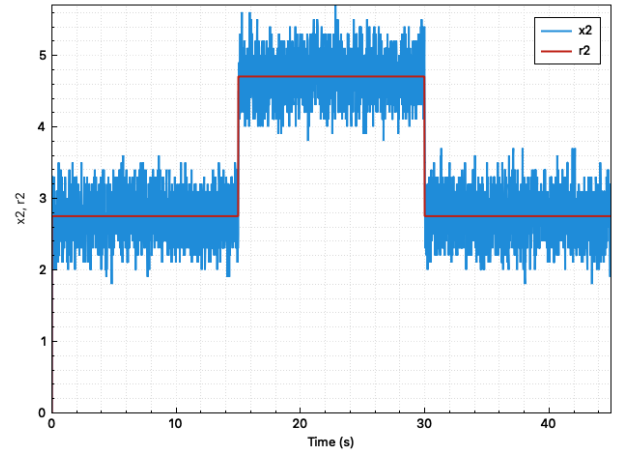
(a) MATLAB (`cstr_state_tracking_attacked.m`), conversion x_1 .



(b) MATLAB (`cstr_state_tracking_attacked.m`), temperature x_2 .



(c) C++ (`cstr_state_tracking_attacked`), conversion x_1 .



(d) C++ (`cstr_state_tracking_attacked`), temperature x_2 .

Fig. 2: Attacked CSTR state trajectories under additive Gaussian noise ($\sigma = 2$). The top row shows the MATLAB reference implementation, the bottom row shows the C++ port, the left column compares x_1 against r_1 , and the right column compares x_2 against r_2 . Both implementations show degraded tracking performance once the stochastic perturbation is introduced.

References

1. P. Tymoshchuk, "A tracking control neural network for the known affine in the inputs discrete-time nonlinear systems," University of North Texas, Tech. Rep., 2024.
2. A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," 2019. [Online]. Available: <https://arxiv.org/abs/1706.06083>
3. H. Hajari, M. Cesaire, L. Schott, S. Lamprier, and P. Gallinari, "Neural adversarial attacks with random noises," *International Journal on*, vol. 32, no. 5, p. 22, August 2023.

A Code Migration: MATLAB to C++

A.1 Motivation

The original tracking control simulation was implemented as a collection of MATLAB scripts (`cstr_state_tracking.m`–`cstr_tracking_error.m`), following the reference implementation in [1]. While MATLAB provides rapid prototyping through built-in matrix operations and plotting, several factors motivated a migration to C++:

1. **Execution speed.** The simulation loop iterates 45,000 time steps with per-step nonlinear dynamics evaluation. C++ compiled code with direct memory access eliminates the interpreter overhead inherent to MATLAB’s execution model.
2. **Memory control.** C++ provides explicit control over stack versus heap allocation, cache-aligned data layouts, and zero-copy data access through pointers—critical when storing $2 \times 45,000$ state matrices.

A.2 Architecture Decisions

The MATLAB implementation uses a procedural style: separate 1-D arrays for each state variable (x , y , r_1 , r_2 , e_1 , e_2 , u) with a single `for` loop driving the simulation. The C++ port uses 2d arrays for the state variables with a single `for` loop as well. It adopts a struct-based architecture that groups related data and separates concerns across compilation units.

A.2.1 Data Layout

Table 1: Data structure mapping between MATLAB and C++ implementations.

MATLAB Variable	C++ Equivalent	Type
$x(1:n)$, $y(1:n)$	<code>stateMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$r_1(1:n)$, $r_2(1:n)$	<code>referenceMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$e_1(1:n)$, $e_2(1:n)$	<code>x1ErrorMatrix</code> , <code>x2ErrorMatrix</code>	<code>Matrix<double, 2, Dynamic></code>
$r_2(i+1)-r_2(i)$	<code>deltaReference</code>	<code>Matrix<double, 2, Dynamic></code>
α , β , ...	<code>InternalConditions</code>	<code>struct</code>
n , $t_{\max}$, dt	<code>SimSettings</code>	<code>struct</code>

The key design choice is consolidating per-variable 1-D arrays into $2 \times N$ Eigen matrices. Row 0 stores the conversion state x_1 , and row 1 stores the temperature state x_2 . This layout preserves temporal locality: accessing both states at time step k requires reading two adjacent memory locations in the same column.

A.2.2 Build System

The C++ project uses CMake 3.15+ with `FetchContent` to manage two external dependencies:

Table 2: C++ build dependencies fetched at configure time.

Library	Source	Version	Purpose
Eigen	GitLab	master	Matrix algebra
cxxplot	US Naval Research Lab	v0.4.1	Interactive Qt-based plotting

The build system produces four executables from separate translation units: `cstr_state_tracking`, `cstr_control_input`, `cstr_tracking_error` (CSTR system under normal operation) and `cstr_state_tracking_attacked` (CSTR system under adversarial attack). Each executable contains its own `main()` function, with shared simulation logic factored into `cstr_dynamics.cpp`.

A.3 Implementation — CSTR System (Fig. 2)

This section presents side-by-side comparisons of the MATLAB and C++ implementations for each major component of the CSTR simulation.

A.3.1 System Dynamics

The CSTR system has two states: conversion x_1 and temperature x_2 . Since the control gain $g_1 = 0$, the conversion state evolves under open-loop natural dynamics. The temperature state uses feedback-linearizing sliding-mode control.

Conversion state x_1 — *open loop*. The natural dynamics follow the discretized CSTR model:

$$x_1(k+1) = x_1(k) + \tau \left[-\alpha x_1(k) + D_a(1 - x_1(k)) \exp\left(\frac{x_2(k)}{1 + x_2(k)/\gamma}\right) \right] \quad (7)$$

MATLAB (`cstr_state_tracking.m`, line 34):

```
1 g2(i)=lambda;
```

C++ (`cstr_dynamics.cpp`, lines 24–29):

```
1     *prevX1 +
2     settings.tau *
3     (-internalConditions.alpha * *prevX1 +
4     internalConditions.Da * (1 - *prevX1) *
5     exp(*prevX2 / (1 + *prevX2 / internalConditions.gamma)));
```

The C++ version accesses parameters through the `InternalConditions` struct rather than workspace variables, and uses cached pointer variables (`prevX1`, `prevX2`) to avoid repeated matrix indexing within the inner loop.

Temperature state x_2 — *closed loop*. The feedback-linearizing control law cancels the nonlinear dynamics $f_2(x)$, yielding the simplified closed-loop update:

$$x_2(k+1) = x_2(k) + \tau[-\beta \operatorname{sgn}(e_2(k)) + \Delta r_2(k)] \quad (8)$$

where $e_2(k) = x_2(k) - r_2(k)$ is the tracking error and $\Delta r_2(k) = r_2(k+1) - r_2(k)$ is the reference change.

MATLAB (`cstr_state_tracking.m`, line 68):

C++ (`cstr_dynamics.cpp`, lines 32–35):

```
1     stateConditions.stateMatrix(1, i) =
2     *prevX2 +
3     settings.tau * (-internalConditions.beta * sgn(e2) + *deltaR2);
```

A notable difference is that the C++ implementation pre-computes the Δr_2 matrix in a single Eigen operation before entering the simulation loop, whereas MATLAB computes $r_2(i+1)-r_2(i)$ inline at each step.

A.3.2 Reference Trajectory

The piecewise-constant reference switches between two operating points at $t = 15$ s and $t = 30$ s.

MATLAB uses time-based conditionals:

```
1 t(i+1)=t(i)+dt;
2
3 if 0<=t(i+1) & t(i+1)<=15
4     r1(i+1)=0.4472;
5 elseif 15<t(i+1) & t(i+1)<30
6     r1(i+1)=0.7646;
7 elseif 30<=t(i+1) & t(i+1)<tmax
```

C++ uses index-based ternary expressions:

```
1     i > 15000 && i < 30000 ? 4.7052 : 2.7520;
2     x1RefVector.push_back(i > 15000 && i < 30000 ? 0.7646 : 0.4472);
```

The index-based approach is valid because $t = i \cdot \tau$ and $\tau = 45/45000 = 0.001$ s, so $t = 15$ corresponds to $i = 15,000$. Additionally, the C++ version pre-computes the reference difference matrix ΔR via Eigen column operations:

```
1
2     const int numRuns = 1;
3     Eigen::MatrixXd stateSum = Eigen::MatrixXd::Zero(2, settings.timeSteps);
```

A.3.3 Signum Function

The signum function $\operatorname{sgn}(\cdot)$ is central to the sliding-mode controller.

MATLAB implements it with explicit branching:

```
1     end
2
3 if (y(i)-r2(i))>0
4     S(i)=1;
5 elseif (y(i)-r2(i))<0
6     S(i)=-1;
7 else
```

C++ uses a branchless template:

```
1     return (T(0) < val) - (val < T(0));
2 }
```

The C++ implementation avoids conditional branching by exploiting boolean arithmetic: the expression evaluates to $+1$, -1 , or 0 without any `if/else` statements, which can improve pipeline utilization on modern processors.

A.3.4 Visualization

MATLAB's native `subplot/plot` functions were replaced with the `cxxplot` library from the US Naval Research Laboratory. The library provides interactive Qt-based plot windows with a named-parameter API:

```

1  downsample(timeVector, x2StateVector, tX2, x2Ds);
2  downsample(timeVector, x2RefVector, tX2Ref, x2RefDs);
3
4  // Plot 1: Conversion (x1)

```

To handle the 45,000 data points efficiently, a min/max envelope downsampling utility reduces the data to approximately 2,000 points while preserving visual fidelity of peaks and transients.

[Table 3](#) maps the MATLAB plotting calls to their C++ equivalents.

Table 3: Visualization API mapping.

MATLAB	C++ (cxxplot)
<code>plot(t, x, 'k-')</code>	<code>cxxplot::plot(t, x)</code>
<code>xlabel('Time (s)')</code>	<code>xlabel_ = "Time (s)"</code>
<code>ylabel('x_1, r_1')</code>	<code>ylabel_ = "x1, r1"</code>
<code>legend('x_1', 'r_1')</code>	<code>show_legend_ = true, name_ = ...</code>
<code>subplot(2,1,k)</code>	separate <code>cxxplot::plot()</code> call per window

A.4 Implementation — Attacked CSTR System

The adversarial attack simulation extends the normal CSTR dynamics by injecting additive Gaussian noise into the temperature state update at each time step. The attack is implemented in both MATLAB (`cstr_state_tracking_attacked.m`) and C++ (`cstr_state_tracking_attacked.cpp`), with the core attacked dynamics defined in the shared `attackedSystemFunction()` in `cstr_dynamics.cpp`.

A.4.1 Noise Injection

The key difference from the normal system is the addition of a stochastic perturbation $\epsilon(k) \sim \mathcal{N}(0, \sigma^2 \Delta t)$ to the temperature state. **MATLAB** (`cstr_state_tracking_attacked.m`, lines 31–37):

```

1  m(i+1)=m(i)+1;

```

```

37 g2(i)=lambda;

```

In MATLAB, the noise sample is drawn via `randn()` scaled by $\sigma \Delta t$, and is added only to the attacked x_2 update.

C++ (`cstr_dynamics.cpp`, lines 46–73):

```

1  std::default_random_engine randomNumberGenerator;

```

```

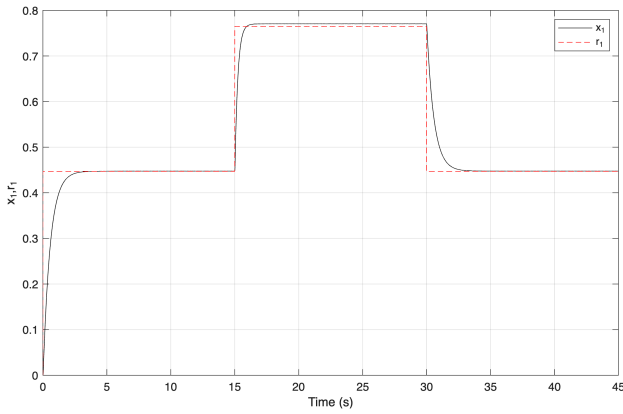
70  *prevX2 + settings.tau * (-internalConditions.beta *
71                          sgn(e2 + epsilon(randomNumberGenerator)) +
72                          *deltaR2);

```

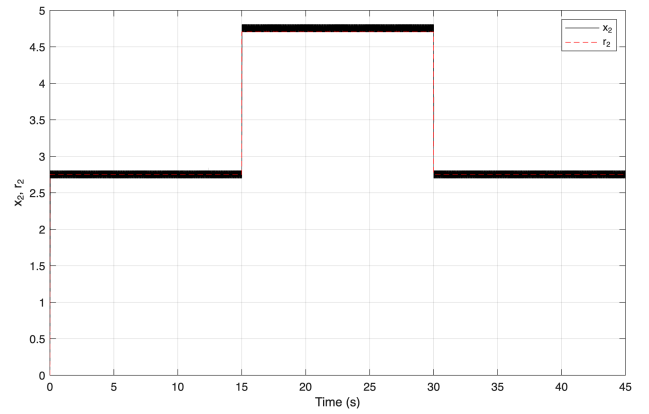
The C++ implementation uses the `<random>` standard library with a `std::normal_distribution` of standard deviation $\sigma = 2$, adding the noise to the x_2 closed-loop update. This follows the perturbed closed-loop formulation in ??.

A.4.2 Output Comparison — CSTR System

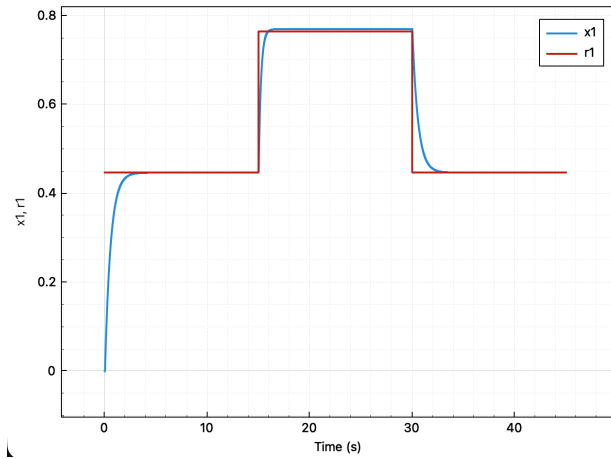
Figure 3 presents a state-aligned comparison of the MATLAB reference implementation and the C++ port.



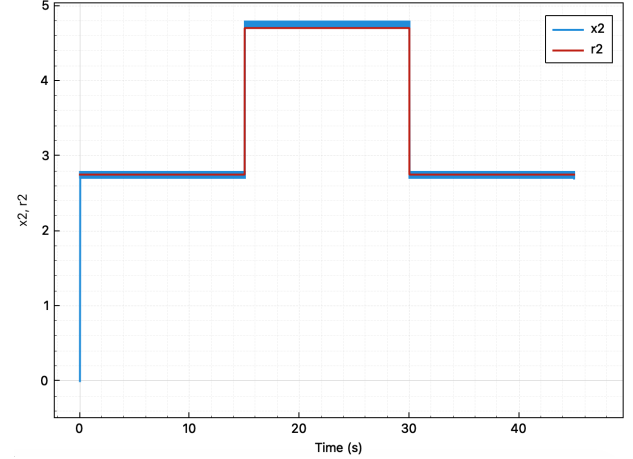
(a) MATLAB (`cstr_state_tracking.m`), conversion x_1 .



(b) MATLAB (`cstr_state_tracking.m`), temperature x_2 .



(c) C++ (`cstr_state_tracking`), conversion x_1 .

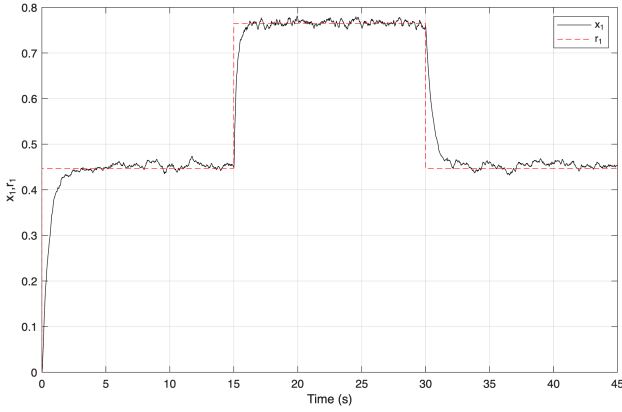


(d) C++ (`cstr_state_tracking`), temperature x_2 .

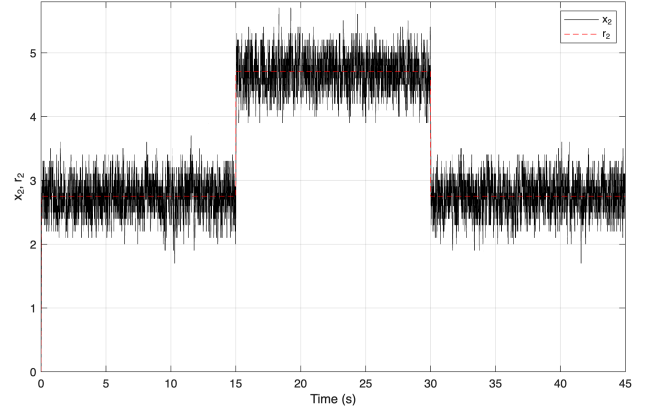
Fig. 3: Comparison of CSTR state trajectories from the MATLAB reference implementation and the C++ port. The top row shows MATLAB, the bottom row shows C++, the left column compares x_1 against r_1 , and the right column compares x_2 against r_2 . Both simulations use identical parameters ($\alpha = 1$, $\beta = 100$, $\lambda = 0.3$, $\gamma = 20$, $D_a = 0.072$) and initial conditions $x(0) = \mathbf{0}$.

A.4.3 Output Comparison — Attacked CSTR System

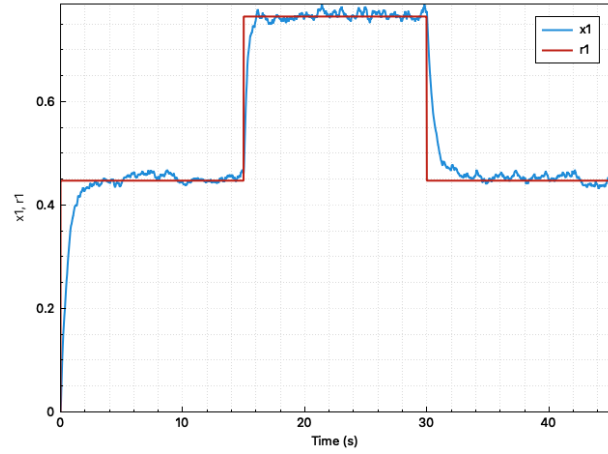
Figure 4 presents the attacked outputs in a state-aligned comparison, with MATLAB and C++ occupying separate rows.



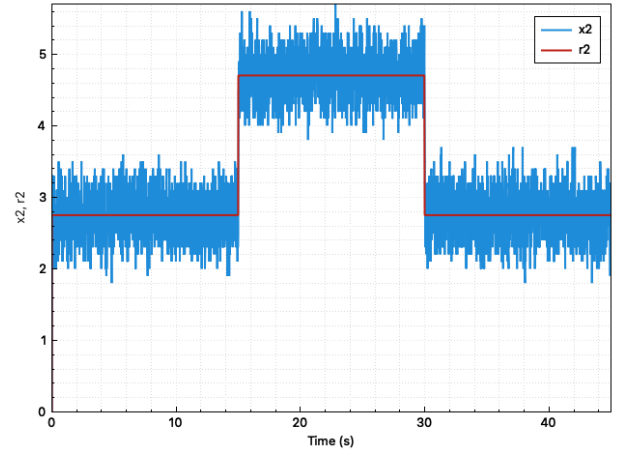
(a) MATLAB (`cstr_state_tracking_attacked.m`), conversion x_1 .



(b) MATLAB (`cstr_state_tracking_attacked.m`), temperature x_2 .



(c) C++ (`cstr_state_tracking_attacked`), conversion x_1 .



(d) C++ (`cstr_state_tracking_attacked`), temperature x_2 .

Fig. 4: Comparison of attacked CSTR state trajectories. Both simulations use $\sigma = 2$ Gaussian noise injection with identical system parameters. The top row shows MATLAB, the bottom row shows C++, the left column compares x_1 against r_1 , and the right column compares x_2 against r_2 . Due to the stochastic nature of the attack, exact numerical equivalence is not expected; instead, the qualitative loss of tracking performance is verified.

Unlike the nominal system (Figure 3), the attacked simulations do not produce identical numerical output because each run draws independent random noise samples. The validation criterion is therefore qualitative: both implementations should exhibit the same degradation pattern in tracking performance under the specified noise intensity.

A.5 Performance Comparison

The C++ simulation includes instrumentation via `std::chrono::high_resolution_clock`. The MATLAB simulation uses `tic/toc`. Table 4 summarises the runtime measurements across both simulation scenarios ($n = 50$ runs each).

Table 4: Runtime comparison between MATLAB and C++ CSTR simulations ($n = 45,000$ steps, $\tau = 0.001$ s, 50 runs each).

Simulation	MATLAB (s)	C++ (s)	Speedup
State tracking (clean)	0.3881 ± 0.1851	0.001690	$229.7\times$
State tracking (attacked)	0.3858 ± 0.1099	0.001495	$258.1\times$

A.6 Output Validation

Verifying numerical equivalence between two independent implementations requires more than visual comparison of plots. The migration includes a CSV-based validation pipeline that compares the C++ simulation output against MATLAB reference data at every time step.

A.6.1 Validation Pipeline

Both implementations export identical 7-column CSV files containing $(t, x_1, x_2, r_1, r_2, e_1, e_2)$ at each of the $n = 45,000$ time steps. The MATLAB script writes its reference data via `writematrix` at the end of the simulation loop; the C++ executable writes its output through `std::ofstream` with 15-digit precision.

A standalone validation executable (`validate_matlab.cpp`) loads both CSV files, extracts corresponding columns into Eigen vectors, and computes the element-wise maximum absolute error for each variable:

$$\varepsilon_{\max}^{(v)} = \max_{k=0}^{n-1} |v_{\text{C++}}(k) - v_{\text{MATLAB}}(k)|, \quad v \in \{x_1, x_2, e_1, e_2\} \quad (9)$$

Each variable passes if $\varepsilon_{\max}^{(v)} < \epsilon$ for a configurable tolerance ϵ . The validator reports per-variable pass/fail status and returns a nonzero exit code on any failure, enabling integration into automated build pipelines.

A.6.2 Validation Results

After correcting an off-by-one initialization error in the reference trajectory (see subsection A.3.2), the validation confirms machine-precision agreement between the two implementations:

Table 5: Maximum absolute error between C++ and MATLAB outputs for the CSTR tracking control simulation ($n = 45,000$, $\tau = 0.001$ s).

Variable	$\varepsilon_{\max}$	Status
x_1 (conversion)	$\sim 10^{-15}$	PASS
x_2 (temperature)	$\sim 10^{-15}$	PASS
e_1 (tracking error)	$\sim 10^{-15}$	PASS
e_2 (tracking error)	$\sim 10^{-15}$	PASS

The errors at $\mathcal{O}(10^{-15})$ are consistent with IEEE 754 double-precision machine epsilon ($\approx 2.22 \times 10^{-16}$), confirming that the two implementations compute identical floating-point results up to rounding in the final digits. The default tolerance of $\epsilon = 10^{-6}$ provides a margin of nine orders of magnitude above the observed error floor.

A.7 Remaining Work

The adversarial attack simulation has been successfully migrated to C++. The following components remain for future work:

1. **Design Defense Mechanism**
2. **Model Defense Mechanism**
3. **Diagram Defense Mechanism**
4. **Simulate Defense Mechanism**